DEVOPS CAPSTONE PROJECT

Taskr

One code change, followed end to end — tested, scanned, versioned, deployed to Kubernetes, observed, and torn back down to nothing.

REPOSITORY

github.com/ChampDeepak/todo

APPLICATION

React 18 · FastAPI · PostgreSQL 16

PLATFORM

Docker · GitHub Actions · Trivy · Terraform · EKS · Helm · Prometheus · Grafana

MODULES 1 & 2 · APPLICATION AND TESTING

A small app, built to be operated

Three tiers, eight endpoints

- **Frontend**
  React 18 + Vite SPA, served by Nginx from a multi-stage build.
- **Backend**
  FastAPI with SQLAlchemy; `GET/POST/PUT/DELETE /api/tasks`, plus `/stats` and `/{id}`.
- **Database**
  PostgreSQL 16, schema managed by Alembic migrations.
- **Health & readiness**
  `/health` is liveness; `/health/ready` checks the database.
- **Metrics**
  `/metrics` exposed via prometheus-fastapi-instrumentator.

Tests as the first gate

32
tests passing

5
test modules

0.4s
full run

- **Isolated per test**
  Each test gets its own database, so order never changes the result.
- **Coverage across the API**
  CRUD, filters, listing, validation and health — not just the happy path.
- **Lint in the same gate**
  Ruff runs beside pytest; style failures stop the build too.
- **Nothing is built if a test fails**
  The image job depends on the test job, so a red test never produces an artifact.

MODULES 3 & 4 · VERSION CONTROL AND CONTAINERS

Reproducible from a clone

Git discipline

- **29 commits, each a real change**
  Messages describe intent, not "update" or "fix".
- **Secrets never committed**
  `.gitignore` excludes `.env`, state files and kubeconfigs.
- **Credentials-free example**
  `terraform.tfvars.example` ships settings without keys.

Images built small and unprivileged

- **Multi-stage frontend**
  `node:22-alpine` builds; `nginx:1.29-alpine` serves. Build tools never ship.
- **Slim backend**
  `python:3.12-slim` with pinned, lock-resolved dependencies.
- **Non-root at runtime**
  Both images drop to an unprivileged user before the process starts.

One command locally

- **`docker compose up --build`**
  Frontend, backend and Postgres, wired together.
- **Health-gated startup**
  The API waits for the database to report healthy rather than racing it.
- **Migrations on boot**
  The entrypoint applies Alembic migrations before serving.

MODULE 5 · CONTINUOUS DELIVERY

The pipeline is the only path to production

git push
main branch

→

Lint & test
Ruff + 32 pytest

→

Build
frontend + both images

→

Trivy scan
HIGH/CRITICAL gate

→

Push to GHCR
tagged by commit SHA

Four jobs, explicit dependencies

- **Lint and test**
  Runs against a real PostgreSQL service container.
- **Build frontend**
  Runs in parallel; independent of the tests.
- **Build, scan and push ×2**
  A matrix over backend and frontend. Reached only if the gates pass.
- **Scan before push, not after**
  The image is built locally, scanned, and published only on a clean result.

Tagged by commit, never `latest`

- **One identifier end to end**
  Commit, CI run, image and running pod all carry the same SHA.
- **Example**
  `ghcr.io/champdeepak/todo-frontend:sha-28d10a4…`
- **Rollback becomes trivial**
  A previous version always exists to return to — with `latest` it does not.
- **Provenance**
  Any running container can be traced back to the exact source that produced it.

MODULE 6 · DEVSECOPS

The scan caught something real

> Trivy runs on every build with `severity: HIGH,CRITICAL` and `exit-code: 1`. `ignore-unfixed` is on, so the gate blocks only on findings someone can actually act on — a scanner that cries wolf gets switched off.

What it found

- **43 findings in the frontend image**
  41 HIGH and 2 CRITICAL, nearly all in OpenSSL.
- **CVE-2026-31789 — CRITICAL**
  Heap buffer overflow in OpenSSL, fixed upstream in 3.3.7.
- **Root cause: an aged base image**
  `nginx:1.27-alpine` shipped Alpine 3.21 with OpenSSL 3.3.3.
- **The push was blocked**
  The vulnerable image never reached the registry.

What changed

- **Base image moved forward**
  `nginx:1.29-alpine` — Alpine 3.23, OpenSSL 3.5.6.
- **Scan now clean**
  0 vulnerabilities across 71 OS packages; secret scanning passes too.
- **Visible in history**
  Commit `d1fb04c` exists for no other reason than this fix.
- **The lesson**
  The flaw was not in code we wrote. It was in a dependency that aged — which is why the scan runs on every build, not once at review.

MODULE 7 · INFRASTRUCTURE AS CODE

Fifteen resources, created and destroyed from code

What Terraform provisions

15
resources

2
availability zones

1.31
Kubernetes version

- **Network**
  VPC `10.0.0.0/16` in ap-south-1, two public subnets across `1a` and `1b`, internet gateway and routing.
- **Cluster**
  EKS control plane with API-mode access and audit logging enabled.
- **Compute**
  Managed node group, two `t3.small` nodes, one per zone.
- **Identity**
  Separate roles for the control plane and the nodes, least privilege by attachment.

Operating it safely

- **Destroy order is derived, not written**
  Node group before cluster, cluster before subnets, subnets before VPC — all from the dependency graph.
- **Proof of reproducibility**
  `Destroy complete! 15 resources destroyed`, then `Plan: 15 to add` against the empty state.
- **No credentials in the repo**
  Auth comes from the AWS CLI profile; state and tfvars are gitignored.
- **Cost discipline**
  EKS bills $0.10/hour whether or not a pod runs, so teardown is part of the workflow.

Deployed by chart, not by hand

The chart

- **`taskr` 0.1.0**
  Templates for backend, frontend, Postgres, ingress and shared helpers.
- **Image tag is a parameter**
  `--set frontend.image.tag=$SHA` is the only thing that changes between releases.
- **Revision history**
  Helm keeps every release, so `helm rollback` is one command.

Workloads

- **Two replicas each**
  Backend and frontend both run two pods.
- **Rolling updates**
  One pod at a time; the readiness probe must pass before the old pod goes.
- **ClusterIP services**
  Backend on 8000, frontend on 8080, Postgres on 5432 — none exposed directly.

Routing

- **Single entry point**
  One Ingress fronts the whole application.
- **`/` → frontend**
  Static assets served by Nginx.
- **`/api` and `/health` → backend**
  Same origin, so no CORS in the browser path.

MODULE 9 · OBSERVABILITY

Metrics from the pods, not the service

FastAPI
exposes /metrics

→

ServiceMonitor
selects backend pods

→

Prometheus
scrapes each pod

→

Grafana
six live panels

What is scraped

- **Both replicas, individually**
  The target list shows 2 / 2 up, each pod by name — not one aggregated service endpoint.
- **Why that matters**
  If one replica degrades, you can see which one. Scraping the Service would hide it behind a load balancer.
- **Kubernetes-native discovery**
  A ServiceMonitor CRD selects by label, so new pods are picked up automatically.

What the dashboard shows

- **Rate**
  Requests per second, broken down by endpoint.
- **Errors**
  Responses by status class — 2xx against 4xx.
- **Duration**
  p50, p95 and p99 latency from histogram buckets.
- **Saturation and health**
  Resident memory per pod, and a live count of pods being scraped.

WHAT ACTUALLY WENT WRONG

Three failures worth keeping

A silent cloud quota

- **Symptom**
  The node group sat for 25 minutes with zero instances and no error on the resource itself.
- **Cause**
  A Free Tier account refuses any instance type not on its list. `t3.medium` is not.
- **Lesson**
  The real reason was only in the Auto Scaling group's activity history. Read the layer that actually launches things.

An interrupted destroy

- **Symptom**
  A rebuilt node group failed: instances ran, but never joined the cluster.
- **Cause**
  A cancelled `terraform destroy` had already deleted the route table in parallel, leaving subnets with no route out.
- **Lesson**
  Interrupting a destroy is not an undo. What survives depends on the graph, not on intent.

A stale base image

- **Symptom**
  CI went red on an image whose application code had not changed.
- **Cause**
  A critical OpenSSL CVE disclosed against the pinned Alpine base.
- **Lesson**
  Dependencies rot without being touched. Scanning every build is what makes that visible.

What this demonstrates

Every gate is automatic

- Tests, lint and a vulnerability scan all run before an artifact exists. Nothing reaches the registry on a human's say-so.

Every artifact is traceable

- The commit, the CI run, the image tag and the running pod share one identifier, which is what makes a rollback a single command.

The environment is disposable

- Fifteen resources destroyed, then fifteen planned to add from the same configuration. Nothing was clicked, and nothing was left billing.

Known shortcuts, stated plainly

- Postgres runs in-cluster rather than RDS, migrations run in the entrypoint rather than a Job, subnets are public to avoid NAT cost, and state is local. Each is a deliberate trade, documented in the README.